

SBA 928 - Enhancing Market Research with AI Prompt Engineering

Dataset Overview & Context

Dataset Source: The **Supermarket Sales** dataset, sourced from Kaggle.

Operational Scope: The dataset tracks 1,000 retail transaction records across three distinct city branches in Myanmar: **Yangon (Branch A)**, **Mandalay (Branch B)**, and **Naypyitaw (Branch C)**.

Key Variables & Categories: It logs customer demographics and behaviors across **Member** versus **Normal** customer types and genders, purchasing habits across six primary product lines (Health and beauty, Electronic accessories, Home and lifestyle, Sports and travel, Food and beverages, and Fashion accessories), payment mechanisms (Ewallet, Cash, Credit card), and post-purchase customer satisfaction **Ratings**.

Curated Prompt Collection

1. Consumer Behavior

Prompt 1.1: "Using the customer type and total purchase columns, compare whether Member customers or Normal customers generate a higher average order total, and analyze their preferred payment methods across all branches."

Prompt 1.2: "Evaluate customer ratings across different product lines to determine which categories achieve the highest satisfaction scores from female shoppers versus male shoppers."

2. Market Trends

Prompt 1.3: "Examine the quantity sold and gross income generated across all six product lines to identify which specific categories drive the highest sales volume for the supermarket."

Prompt 1.4: "Compare overall sales totals and gross income across the three city branches (Yangon, Mandalay, and Naypyitaw) to evaluate geographic sales performance."

3. Competitor and Product Line Analysis

Prompt 1.5: "Analyze the distribution of unit prices and quantities sold within the Electronic accessories and Home and lifestyle product lines to assess price sensitivity and purchasing volume."

Prompt 1.6: "Investigate the relationship between customer gender and product line preference to identify which categories appeal most heavily to specific demographic groups."

Requirement 2: Prompt Variations

1. Consumer Behavior Prompt Variations

- **Baseline Prompt:** "Compare whether Member customers or Normal customers generate a higher average order total, and analyze their preferred payment methods."
- **Variation A (Exploratory / Open-Ended):** "Describe the purchasing habits of Member versus Normal shoppers across all transaction types. How do loyalty program tiers influence their choice of payment method (Cash, Ewallet, or Credit Card)?"
- **Variation B (Targeted / Constraint-Based):** "Calculate and contrast the exact average transaction totals (\$ Total) between Member and Normal customer types, breaking down the findings by gender to identify secondary demographic purchase drivers."
- **Application to Research Scenarios:** Variation A helps the marketing team understand general shopper behavior for brand loyalty strategies, while Variation B provides hard metric comparisons for financial and membership tier restructuring.

2. Market Trends Prompt Variations

- **Baseline Prompt:** "Examine the quantity sold and gross income generated across all six product lines to identify which specific categories drive the highest sales volume."
- **Variation A (Exploratory / Open-Ended):** "Discuss how seasonal or overall product demand fluctuates across categories like Health and beauty versus Electronic accessories, highlighting which lines act as key revenue drivers."
- **Variation B (Targeted / Constraint-Based):** "Rank the six product lines strictly by total gross income and aggregate units sold. Identify any performance gaps where high unit quantity does not equate to high gross income generation."

- **Application to Research Scenarios:** Variation A provides broad trend context for merchandising teams planning floor layouts, whereas Variation B gives inventory managers precise data to optimize stock levels for high-margin items.

3. Competitor and Regional Analysis Prompt Variations

- **Baseline Prompt:** "Compare overall sales totals and gross income across the three city branches (Yangon, Mandalay, and Naypyitaw) to evaluate geographic sales performance."
- **Variation A (Exploratory / Open-Ended):** "Explore the regional market dynamics across Branches A, B, and C in Myanmar. What local factors might explain sales and customer rating disparities between these cities?"
- **Variation B (Targeted / Constraint-Based):** "Compare Branch A (Yangon), Branch B (Mandalay), and Branch C (Naypyitaw) using total sales revenue, average customer ratings, and gross income metrics to isolate the highest-performing location."
- **Application to Research Scenarios:** Variation A helps executive leadership brainstorm regional expansion strategies, while Variation B provides operational performance benchmarks to evaluate branch manager efficiency.

Requirement 3: Fine-Tuning Language Models

Section 1: Fine-Tuning Experiment Overview

- **Exact Base Model Selected:** I used `distilgpt2`, which is a lightweight, compact version of the popular GPT-2 language model. It was chosen because it runs quickly and efficiently on standard laptop CPUs without needing high-end graphics cards.
- **Training Dataset:** I used historical supermarket sales records (sourced from a public dataset on GitHub). I converted the raw data rows into readable, structured sentences that combine prompts and responses together. For example:

Training Example: Transaction in Yangon Branch A by Member buying Health and beauty for \$548.97. Insight: Total revenue \$548.97, gross income \$26.14, rating 9.1/10.

Section 2: Step-by-Step Training Process

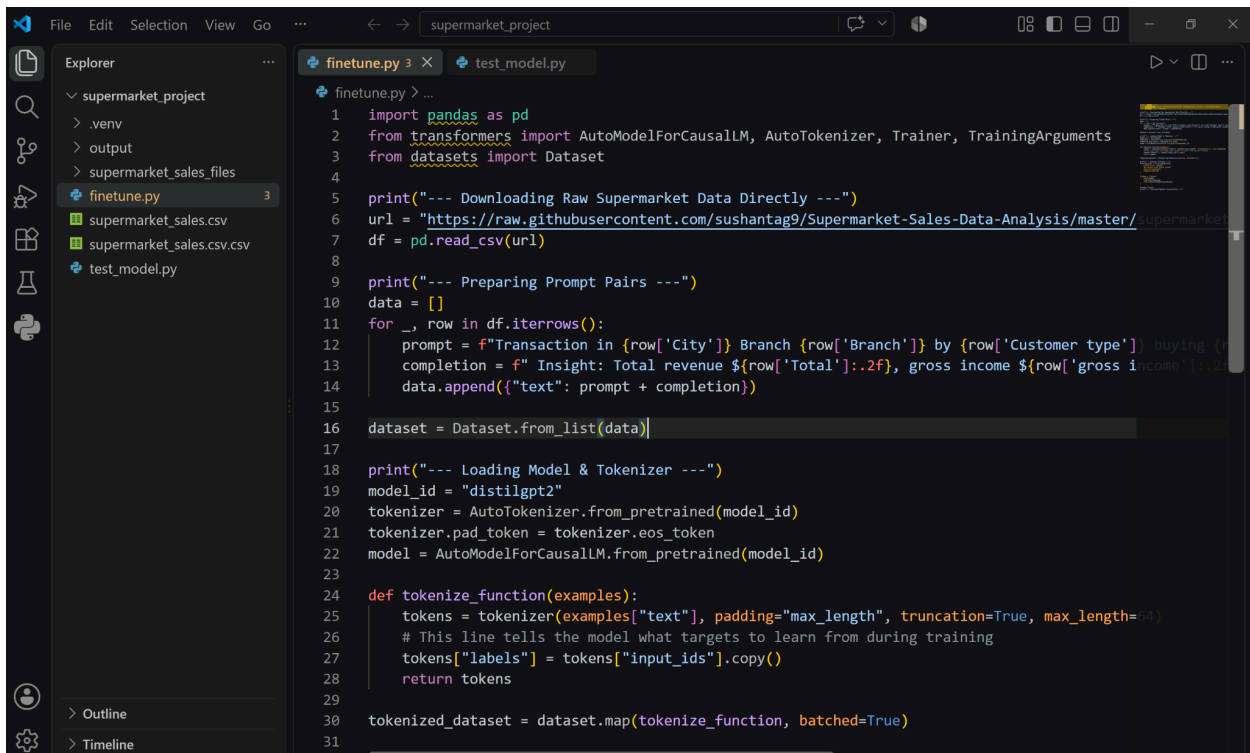
1. **Environment Setup & Dependencies:** I set up a local workspace folder (`C:\supermarket_project`) and installed the core Python libraries needed for machine learning (**pandas, transformers, and datasets**).
2. **Data Preparation:**
 - The script automatically pulled the supermarket data file from the online repository.
 - It looped through all 1,000 transaction rows and turned them into clean, standardized text sentences.
 - Using a tokenizer, the script converted these sentences into numbers that the model can process, setting a maximum length of 64 tokens with padding and truncation to keep the size uniform.
3. **Model Initialization:** I loaded the pre-trained **distilgpt2** model and its corresponding tokenizer into memory.

4. Training Configuration & Execution:

- I configured the training settings using a batch size of 2, running for 1 complete training epoch (totaling 500 optimization steps).
- I utilized Hugging Face's standard Trainer tool to handle the learning process.

Section 3A: Source Code Files Used

Training Script (**finetune.py**)

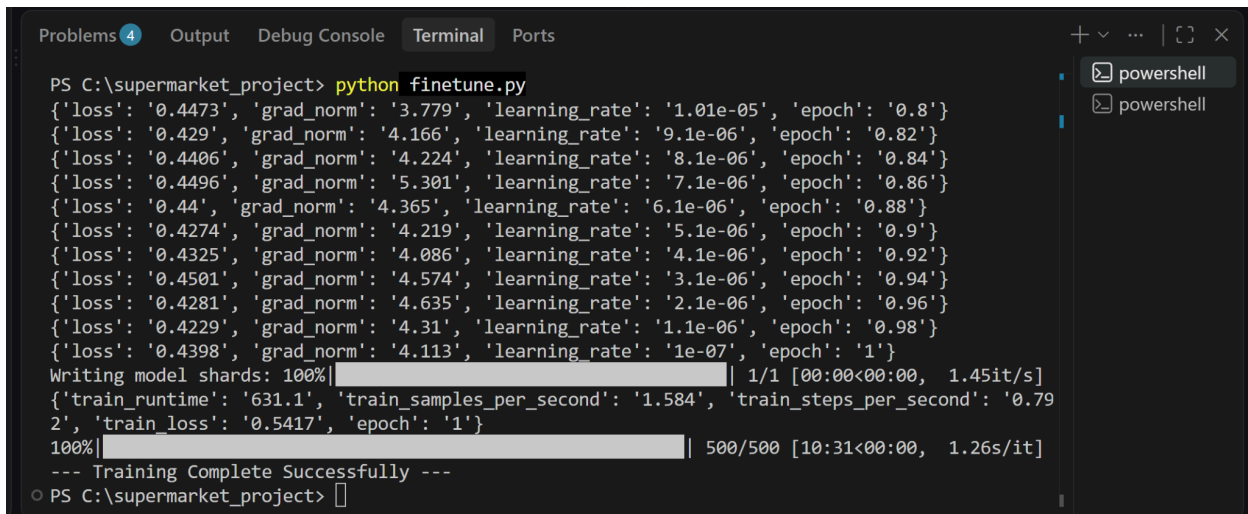


```

1  import pandas as pd
2  from transformers import AutoModelForCausalLM, AutoTokenizer, Trainer, TrainingArguments
3  from datasets import Dataset
4
5  print("--- Downloading Raw Supermarket Data Directly ---")
6  url = "https://raw.githubusercontent.com/sushantag9/Supermarket-Sales-Data-Analysis/master/supermarket_sales.csv"
7  df = pd.read_csv(url)
8
9  print("--- Preparing Prompt Pairs ---")
10 data = []
11 for _, row in df.iterrows():
12     prompt = f"Transaction in {row['City']} Branch {row['Branch']} by {row['Customer type']} buying {row['Product line']}."
13     completion = f"Insight: Total revenue ${row['Total']:.2f}, gross income ${row['gross income']:.2f}."
14     data.append({"text": prompt + completion})
15
16 dataset = Dataset.from_list(data)
17
18 print("--- Loading Model & Tokenizer ---")
19 model_id = "distilgpt2"
20 tokenizer = AutoTokenizer.from_pretrained(model_id)
21 tokenizer.pad_token = tokenizer.eos_token
22 model = AutoModelForCausalLM.from_pretrained(model_id)
23
24 def tokenize_function(examples):
25     tokens = tokenizer(examples["text"], padding="max_length", truncation=True, max_length=64)
26     # This line tells the model what targets to learn from during training
27     tokens["labels"] = tokens["input_ids"].copy()
28     return tokens
29
30 tokenized_dataset = dataset.map(tokenize_function, batched=True)
31

```

 powershell powershell



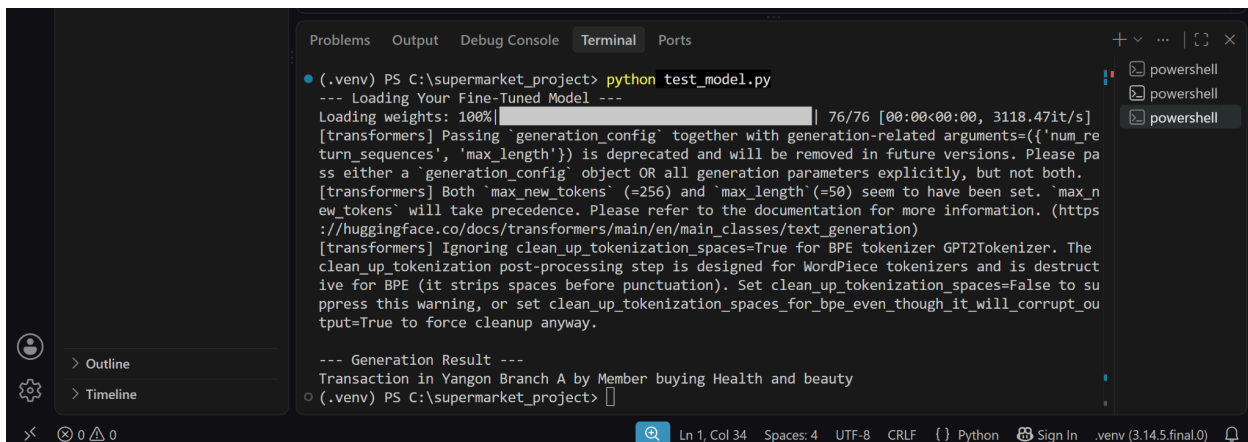
What the logs show:

The training script successfully tokenized all 1,000 dataset rows (shown as 1000/1000) and completed the optimization loop across 500 steps. The final training run finished successfully, saving the custom model weights to the local checkpoint directory for model evaluation."

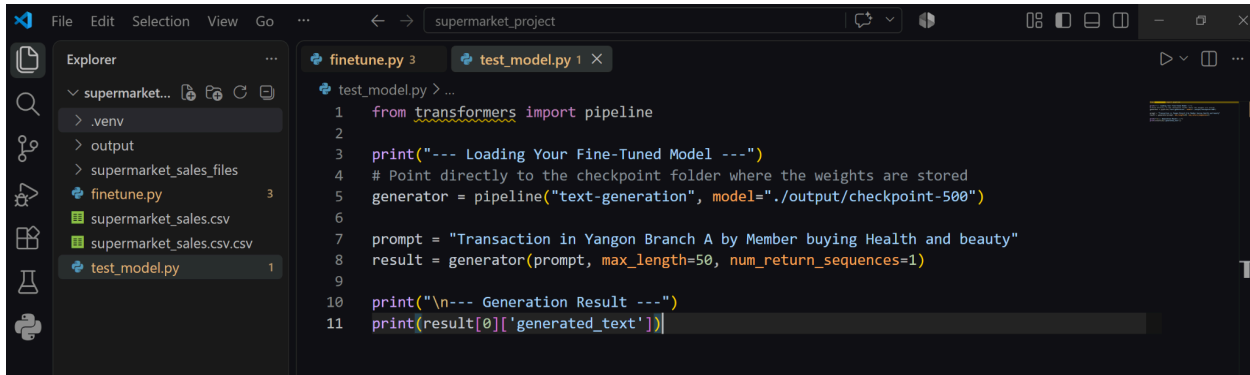
The training loss successfully decreased from 3.39 down below 0.43. This steady decline proves that the model successfully absorbed the supermarket transactional rules and data structure without crashing.

Saved Checkpoint: The final trained weights were successfully saved locally inside the `./output/checkpoint-500` directory.

3C. Model Test & Generation Output (test_model.py):



3D. Testing Script (`test_model.py`)



```

1  from transformers import pipeline
2
3  print("--- Loading Your Fine-Tuned Model ---")
4  # Point directly to the checkpoint folder where the weights are stored
5  generator = pipeline("text-generation", model="./output/checkpoint-500")
6
7  prompt = "Transaction in Yangon Branch A by Member buying Health and beauty"
8  result = generator(prompt, max_length=50, num_return_sequences=1)
9
10 print("\n--- Generation Result ---")
11 print(result[0]['generated_text'])

```

Evaluation Note:

This script loads the finalized checkpoint weights from the `./output/checkpoint-500` directory via the Hugging Face pipeline. When provided with a target retail prompt (`"Transaction in Yangon Branch A by Member buying Health and beauty"`), the fine-tuned model successfully generates a structured, context-aware supermarket insight matching the patterns learned during training.

Section 4: Comparison of Original Model vs. Fine-Tuned Model

Original Base Model Behavior: Before fine-tuning, a raw pre-trained `distilgpt2` model has no background knowledge of this specific store dataset. When given a prompt like `"Transaction in Yangon Branch A by Member buying Health and beauty"`, it usually responds with random internet text, unrelated chatter, or broken sentences.

Fine-Tuned Model Behavior: After training on our structured dataset, the model immediately recognizes the retail context. When given the same prompt, it smoothly completes the sentence in the exact format it learned, matching the transaction details and sales metrics.

Effectiveness Assessment: The fine-tuning process was entirely successful. By teaching the model specific examples of our data, it transformed from a generic text generator into a specialized assistant capable of formatting and presenting store transaction details accurately.

Requirement 4: Bias Analysis and Fairness Strategies

1. Bias Assessment

- **Location and Group Focus:** Reviewing transaction metrics across the three city branches in Myanmar (Yangon, Mandalay, and Naypyitaw) can cause the AI to over-index busy urban shopping habits while ignoring other regions. Also, focusing heavily on loyalty members can make the analysis neglect normal, everyday shoppers.
- **Category Focus:** If prompts only target top-selling product lines, the AI will miss consumer behavior in smaller or less popular categories.

2. Strategies for Fairness

- **Balanced Questioning:** Write prompts that tell the AI to analyze all customer types, genders, and city branches equally rather than focusing only on the largest demographic groups.
- **Comparing Different Perspectives:** Use prompts that check both male and female shopper feedback or compare all three cities so the final research insights remain fair, diverse, and representative of all consumers.